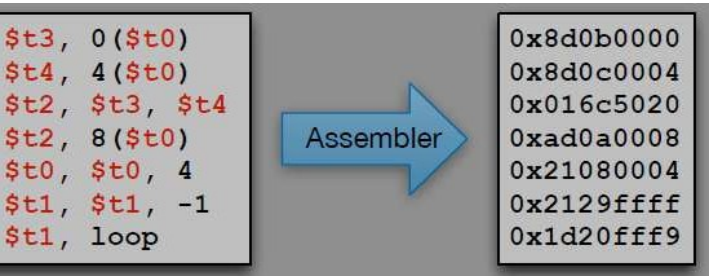
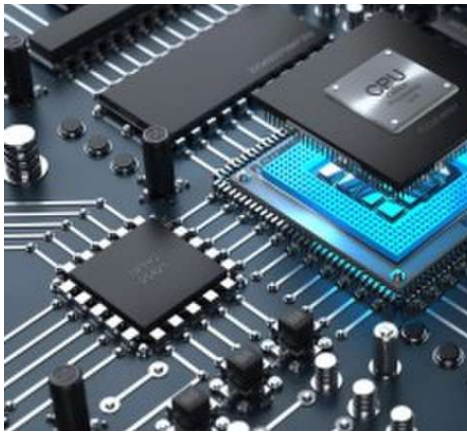




The OS Syscall in MIPS Assembly (Video Recording)

CMPSC 64

ZIAD MATNI, PH.D.

UNIVERSITY OF CALIFORNIA, SANTA BARBARA

```
xt              # We always have to have this starting line – this is called a label
n:    # “main:” is another label – it is also necessary
      li $t0, 5
      li $t1, 7
      add $t2, $t0, $t1


rint the integer that’s in $t2 to std.output
      li $v0, 1
      move $a0, $t2
      syscall


nd program
      li $v0, 10
      syscall
```

Wow… All of this cod
just so we can add 5 t
and display the result

Recall the Role of the Operating System

When you run a program on a computer,
is it running all by itself? Does it have the CPU all to itself?

There's usually several programs (applications) running on a computer at a given moment

You need something to organize the sharing of the computer's resources
(comp. sys. resources: memory, CPU, network connections, I/O, etc…)

The Operating System is a **program** that facilitates easy, efficient, fair, orderly, and secure use of various software and hardware resources

The “System Call”

A program makes a “**system call**” to the OS if it needs to:

- Access I/O (e.g. the std.out or the std.in)
- Ask for more memory, send data over a network link, and so on…
- Example: when a C++ program says `std::cout << "Print this!";`

In other words, the “**System Call**” happens when the program switches control to the OS kernel which will try to process this request

How Does our MIPS Program talk to the OS?

Recall: We are running our code on the MIPS ***emulator*** called **spim**

We're not actually running our commands on an actual MIPS (hardware) process

…we're letting software *pretend* it's hardware…

Ok, so how might we print something onto *std.out*? Or get a value from *std.in*?

- *ANS: We emulate a call to the OS*

spim syscall Routines

MIPS features a `syscall` instruction, which triggers
a ***software interrupt***, or ***exception***

In the real world, syscall **pauses the program** and tells the OS
to go do something with I/O

Inside **spim**,
it tells the emulator to *pretend* the OS is doing something with I/O

syscall

So we have the OS/emulator's attention,
but how does it ***know what we want?***

ANS: We pass on unique codes that we put in specific registers

So… is there a "code book"????

Yes! All CPUs come with manuals.
For us, we have the **MIPS Reference Card**

`syscall` Interaction Setup

You will need:

System call code
- Placed in **$v0**

Argument
- Placed in **$a0**

Execute a system call
- Use the **syscall** instruction

EXAMPLE:

```
# Let's print an integer to std.output
li $v0, 1
# $v0 = 1 is code for:
# "I want to print an integer to std.out

# What are we printing?
# Let's say, whatever's in register $t0.
move $a0, $t0

# Execute!
syscall
```

Is $v0 Strictly Used for `syscall` Codes?

No: **$v0** has other uses too (more on those later!)

BUT, when we use it for **`syscall`**, we make sure we have a code in it

Examples:

syscall function	Value in $v0	What else is used?
Print an integer (std.out)	1	$a0 ← integer to print
Get an integer (std.in)	5	$v0 ← gets the input int
Print a string (std.out)	4	$a0 ← mem address of the string
Quit a program	10	n/a

We'll explore these…

Printing to std.out

```
.text
# Main program
main:
        li $t0, 5
        li $t1, 7
        add $t3, $t0, $t1

# Print the integer that's in $t3 to std.output
# First: set up the requisite registers, then execute a system call!
        li $v0, 1       # Put the immediate value 1 into $v0
        move $a0, $t3   # Move (i.e. make a copy) the value in $t3 into $a0
        syscall         # Print to std.out the integer that's in $a0
```

*ere is not s long as **$v0** set up before call*

Notes on Program Files for MIPS Assembly

The programs you write have to be **text file types**

- Like any other program file…!

Typical file extension type is **.asm**

To leave comments, use **#** at the start of the line

For our assignments, always run these on **spim** on CSIL (lab will demo)

We're Not Quite Done Yet!
Exiting an Assembly Program in SPIM

You ALSO need to say ***when you are done!***

- This is an important communication from the program to the OS (why?)
- Most HLL programs do this for you automatically

How is this done in **spim**?

- Simply issue a `syscall` with a special value in **$v0 = 10**

```
xt              # We always have to have this starting line – this is called a label
n:    # “main:” is another label
      li $t0, 5
      li $t1, 7
      add $t2, $t0, $t1

rint the integer that’s in $t3 to std.output
      li $v0, 1
      move $a0, $t3
      syscall

nd program    --- You should always have these 2 lines at the end of ALL your programs!!!
      li $v0, 10
      syscall
```

Let's Run This Program Already!
Using SPIM

We'll call it **simpleadd.asm**

Run it on CSIL as: `$ spim -f simpleadd.asm`

What About *Getting* an Input (via Std In)?

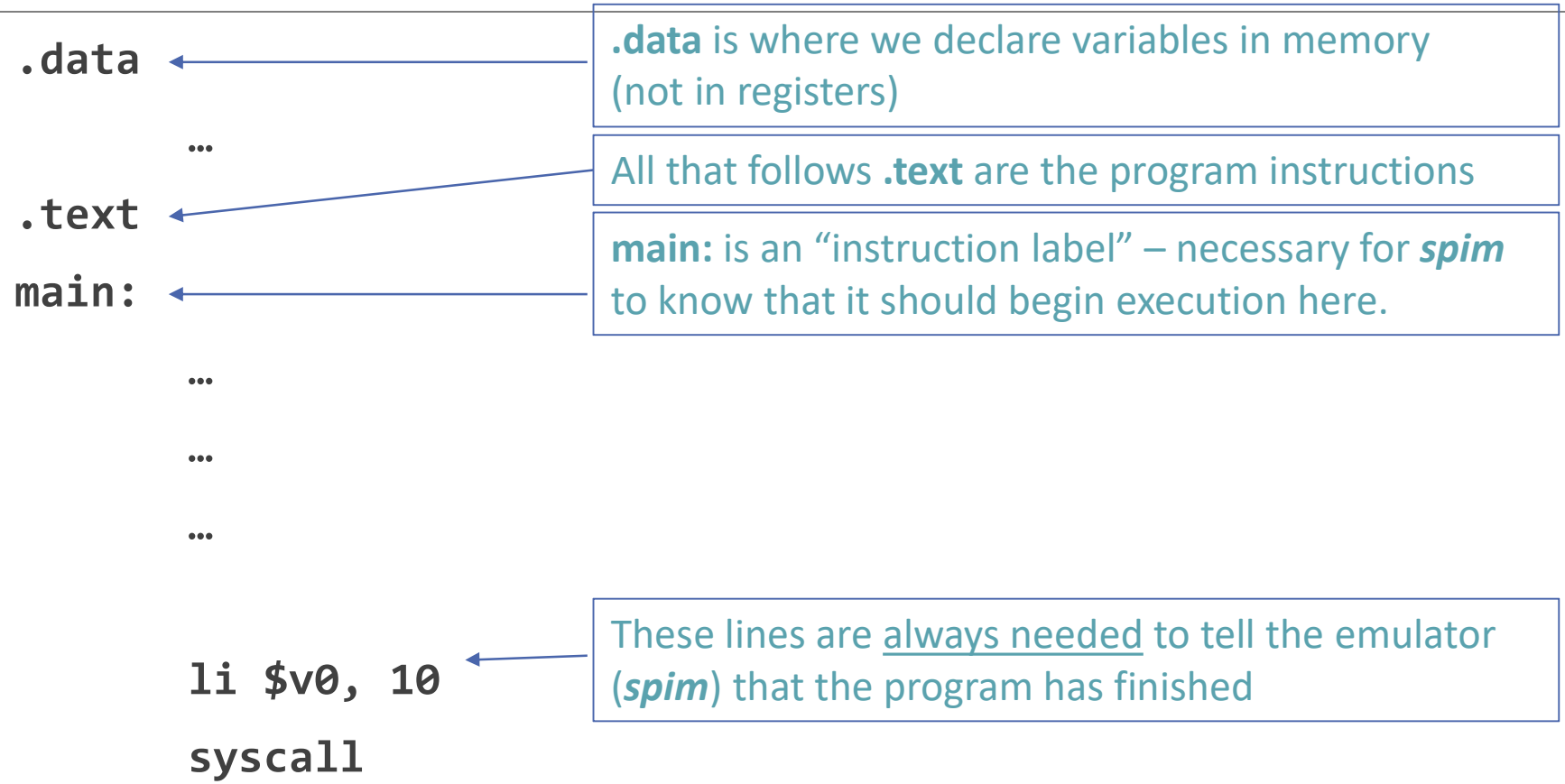
```
# Get an integer value from user, so make $v0 = 5
li $v0, 5
syscall

# Your new input int is now the value in $v0 (over-rode the
# You can move it around and compute with it like normal...
move $t0, $v0
sll $t0, $t0, 2     # Multiply it by 4
add $t0, $t0, $t0   # Add it to itself... etc...
...
```

Another Demo!

We'll call it **simpleInput.asm**

Run it on CSIL as: `$ spim -f simpleInput.asm`

The Proper Format of an Assembly Program



Printing Strings using syscall

```
      # This label is for when we want to pre-define values in memory (not registers)
  mystr: .asciiz "Hello World!\n"  # The address is handled by spim, reference is label “mystr”



nt string to std.output (not an int!! We have to use a different code!)
ing $v0 = 4 tells syscall to expect a string to be printed...
  li $v0, 4
ce a string is an array of characters, and we can’t add the value of the whole array!
 we load the address of that array into $a0 (we’ll use a new instruction here: la)
  la $a0, mystr           # la = load memory address
  syscall

 program
  li $v0, 10
  syscall
```

One More Demo!!

We'll call it **hello.asm**

Run it on CSIL as: `$ spim -f hello.asm`

</LECTURE>